

CSA15 - CLOUD COMPUTING AND BIG DATA ANALYTICS

TEAM ASSIGNMENT RESPONSE TEMPLATE

Field	Student Entry
Slot	B
Assignment Title	Cloud-Ready Smart Campus Service Platform: API Implementation, Workload Sizing and Virtualized Deployment
Team Number	3
Date of Submission	02.09.2026
GitHub Repository	https://github.com/192421053simats-cmd/Cloud-ready-smart-campus-service.git

TYPE DIRECTLY IN THIS DOCUMENT. Replace all bracketed placeholders with original content. Do not reproduce the assignment question, detailed instructions or another team's text.

Team Members

S.No.	Register Number	Student Name	Owned Module / Responsibility
1	192421053	S.Blessy	API implementation and workload sizing
2	192521156	G. Akshaya	Frontend and API integration
3	192511279	P. Krishnapriya	Testing, documentation and deployment evidence
4	[If applicable]	[If applicable]	[If applicable]

1. Problem Interpretation and Team Innovation

Problem Interpretation

The project provides a cloud-ready smart campus service platform that allows students and faculty to access campus services through secure APIs. The solution focuses on reliable service access, controlled resource usage and a deployable virtualized backend. The design connects API implementation with workload sizing so that the selected infrastructure is justified by expected campus usage.

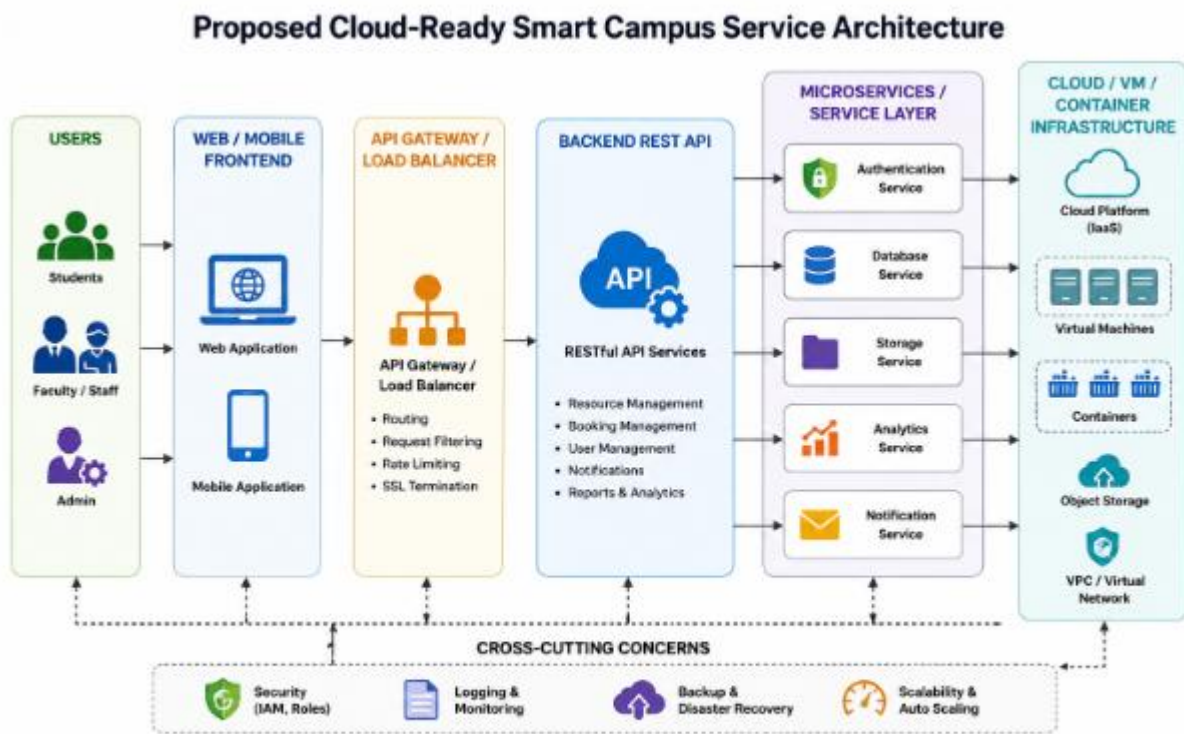
Approved Team Innovation

Cloud-Ready Smart Campus Service Platform. Its purpose is to provide students and faculty with a centralized service interface for campus resources such as room/resource discovery and booking. The measurable value is faster service access, reduced manual coordination and an infrastructure configuration that can be scaled according to measured request load.

Requirement / Constraint	Type	Priority	Acceptance Evidence
Secure REST API authentication	[Functional / Performance / Security]	[Must / Should]	Security
Must	Successful login/token test	Resource search and booking API	Functional
Must	Valid API response and booking test	Peak workload support of ~3.5 req/s	Performance
Must	Load/performance evidence	Restricted VM/network access	Security

2. Architecture, API and Pseudocode

Architecture Diagram



Method	Route	Purpose	Input / Output	Status Codes
Must	Firewall/network configuration screenshot	POST	/api/auth/login	Authenticate user and issue token
Credentials → token	200 / 401	GET	/api/resources	List available campus resources
Filters → resource list	200 / 400	POST	/api/bookings	Create a resource booking
Resource/time → booking	201 / 400 / 409	GET	/api/bookings/{id}	Retrieve booking details

Pseudocode

```

const express = require("express");
const app = express();

app.use(express.json());

// Authentication middleware
function authenticateUser(req, res, next) {
  const token = req.headers.authorization;

  if (!token) {

```

```

        return res.status(401).json({
            success: false,
            message: "Authentication required"
        });
    }

    // Token verification
    req.user = {
        id: "USER001",
        role: "student"
    };

    next();
}

// Authorization middleware
function authorizeRole(roles) {
    return (req, res, next) => {
        if (!roles.includes(req.user.role)) {
            return res.status(403).json({
                success: false,
                message: "Access denied"
            });
        }

        next();
    };
}

// Resource API
app.get("/api/resources",
    authenticateUser,
    authorizeRole(["student", "faculty", "admin"]),
    (req, res) => {

        const resources = [
            {
                id: 1,
                name: "Seminar Hall",
                status: "Available"
            },
            {
                id: 2,
                name: "Computer Lab",
                status: "Available"
            }
        ];

        console.log("Audit: Resource data accessed");

        res.status(200).json({
            success: true,
            resources: resources
        });
    }
);

// Booking API
app.post("/api/bookings",

```

```

authenticateUser,
authorizeRole(["student", "faculty", "admin"]),
(req, res) => {

    const { resourceId, date, startTime, endTime } = req.body;

    // Validate request data
    if (!resourceId || !date || !startTime || !endTime) {
        return res.status(400).json({
            success: false,
            message: "Required booking data is missing"
        });
    }

    // Create booking
    const booking = {
        bookingId: "B001",
        userId: req.user.id,
        resourceId: resourceId,
        date: date,
        startTime: startTime,
        endTime: endTime
    };

    // Record audit information
    console.log("Audit: New booking created", booking);

    res.status(201).json({
        success: true,
        message: "Booking created successfully",
        booking: booking
    });
}

// Get booking details
app.get("/api/bookings/:id",
authenticateUser,
(req, res) => {

    const bookingId = req.params.id;

    if (!bookingId) {
        return res.status(400).json({
            success: false,
            message: "Booking ID is required"
        });
    }

    res.status(200).json({
        success: true,
        bookingId: bookingId,
        message: "Booking details retrieved successfully"
    });
}

// Start server
app.listen(3000, () => {

```

```
console.log("Smart Campus API running on port 3000");  
});
```

3. Infrastructure Sizing and Deployment Decision

Engineering Result	Formula / Method	Substitution	Final Value	Justification
Daily active users	Booking ID → booking	200 / 404	40% of total campus users	$0.4 \times (3000 + 500)$
Peak active users	1400	Not all users access the platform daily	25% of DAU during peak hour	0.25×1400
Peak requests/second	350	Requests cluster before popular campus slots	3 requests per active user in 5 minutes	$(350 \times 3) / 300 \text{ sec}$
Required vCPU	$\approx 3.5 \text{ req/s}$	Conservative peak estimate	$\approx 50 \text{ req/s}$ per vCPU; round up with headroom	$3.5 / 50 + \text{headroom}$
Required RAM	2 vCPU	Small prototype footprint with safety margin	$\approx 512 \text{ MB}$ base + connection/runtime overhead	$512 \text{ MB} + \approx 500 \text{ MB}$
Required storage	$\approx 1 \text{ GB}$; allocate 2 GB	OS and application runtime need practical headroom	Users + resources + bookings + logs for one year	$(3500 \times 2 \text{ KB}) + (50 \times 1 \text{ KB}) + (1400 \times 365 \times 0.5 \text{ KB})$
Growth headroom	$\approx 263 \text{ MB}$ raw; allocate 20 GB	Indexes, OS, logs, backups and growth buffer	30% growth over 3 years	3.5×1.3

Selected Deployment Environment

Selected environment: Ubuntu Server LTS virtual machine with Docker as an optional container packaging layer. A VM is selected because it provides a controlled OS-level deployment environment, clear CPU/RAM/disk allocation and straightforward evidence for virtualization. Docker can package the REST API consistently and supports later migration to a cloud container service.

Security and Recovery Controls

Use token-based authentication, role-based authorization, input validation and HTTPS. Store secrets outside source code using environment variables or a secret manager. Keep the database private, restrict inbound network rules, maintain audit logs and take VM/database backups or snapshots. Use the latest verified backup/snapshot for rollback after a failed deployment.

4. Implementation and Source Code

Module	Technology	Owner	Repository Path	Execution Evidence
≈4.55 req/s	Avoid resizing every semester	Backend REST API	Node.js / Express	S.Blessy
backend/ API test screenshot	Frontend	HTML / CSS / JavaScript	G. Akshaya	frontend/ browser execution screenshot
Database	MongoDB	P. Krishnapriya	database/ CRUD/API evidence	Virtualized deployment
Ubuntu Server LTS + Docker	S.Blessy	deployment/ VM/container evidence	Authentication endpoint	Valid user credentials

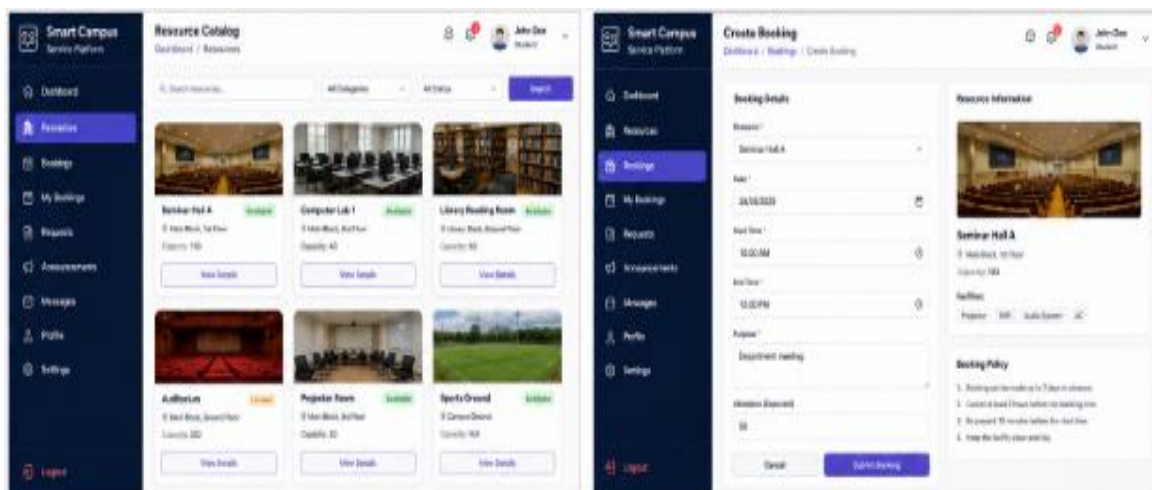
Source Code in Selectable Text

Source code should be pasted here from the team's GitHub repository as selectable text, following the template requirement. Recommended principal implementation: REST API routes, authentication/authorization middleware, resource/booking logic, database access and validation. Do not replace source code with screenshots.

5. Results, Testing and Validation

Test / Requirement	Input or Condition	Expected Result	Observed Result	Evidence
Token returned	Token returned successfully	API test screenshot	Resource search	GET resources
Resource list returned	Valid JSON response	API test screenshot	Booking creation	Valid resource/time
Booking created	Booking ID returned	API test screenshot	VM sizing/deployment	2 vCPU, 2 GB RAM, 20 GB disk
Service runs correctly	Verify on actual VM	VM terminal/config screenshot	https://github.com/192421053simats-cmd/Cloud-ready-smart-campus-service.git	https://github.com/192421053simats-cmd/Cloud-ready-smart-campus-service.git

Result Evidence



Picture 2: Resource catalog / search results

Picture 3: Resource booking flow (create booking)



Picture 4: User profile / settings

Engineering Conclusion

The workload model supports a small prototype deployment because the estimated peak traffic is approximately 3.5 requests/second. A 2-vCPU, 2-GB RAM VM with a 20-GB disk provides practical headroom for the operating system, REST API, logs and test data. The main limitation is that the request-rate assumptions are estimates rather than production measurements. Before production, the team should run load tests, monitor CPU/RAM/latency, add HTTPS and stronger secret management, use managed database/storage services where appropriate, and scale horizontally when measured demand increases.

6. GitHub and Submission Record

Evidence	Repository Path / URL	Verified
Source code	https://github.com/192421053simats-cmd/Cloud-ready-smart-campus-service	[Yes / No]
README and setup	https://github.com/192421053simats-cmd/Cloud-ready-smart-campus-service.git	[Yes / No]
Architecture source	https://github.com/192421053simats-cmd/Cloud-ready-smart-campus-service.git	[Yes / No]
Tests and screenshots	To be completed from actual team evidence	[Yes / No]
Contribution evidence	To be completed from actual team evidence	[Yes / No]

References and Tool Disclosure

References: CSA15 Slot B Team Assignment Response Template and course materials. External/AI assistance: used for drafting and organizing explanations and workload-sizing calculations. Verification: team members must verify all API outputs, VM configuration, screenshots, source code and repository contents against their actual implementation before submission.

FINAL CHECK: Typed Word document; original team content; source code as text; readable evidence; working GitHub link; similarity at or below 20%.